

day1 종합실습

판교 5반 홍은비

1. 시스템 구성



구성 요소	컨테이너	포트 및 연결	역할
Web	board-web	8080:80	게시판 화면 제공
WAS	board-was	5001:5000	API 요청 처리
DB	board-db	board-db:5432	게시글 데이터 저장

- 세 컨테이너는 board-net 에 연결했다.
- WAS에는 DB_HOST , DB_PORT , DB_NAME , DB_USER , DB_PASSWORD 를 환경변수로 전달했다.

3. 실행 결과

3.1 이미지 빌드 및 컨테이너 실행

▼ WAS 빌드·실행 및 로그

```
● → board git:(main) x docker start board-db
board-db
● → board git:(main) x docker build -t board-was:1.2 ./was
[+] Building 1.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 270B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:9c900dea9e8fb7e16277c179b555cc72d29a352dbc33cff48
=> => resolve docker.io/library/python:3.11-slim@sha256:9c900dea9e8fb7e16277c179b555cc72d29a352dbc33cff48
=> [internal] load build context
=> => transferring context: 331B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:e0b98418c3c68501db82bd69ff4d93647d4396ac59fb0dfc4ae7e5065ae849ee
=> => exporting config sha256:0406a834033e565c22aecdl1eb0ba7ebedd01225573447d80190dcf203d2b1f34
=> => exporting attestation manifest sha256:30f5a6c132af56024eeb1babb7ae7f4b30877a54c7b3632d0c52283f113b0
=> => exporting manifest list sha256:bfad4fc18fd9e4ce27f4e67d98804242c0a58cf0e76653125fd153be73ff4847
=> => naming to docker.io/library/board-was:1.2
=> => unpacking to docker.io/library/board-was:1.2
```

WAS 이미지 빌드

```
● → board git:(main) x docker rm -f board-was
board-was
● → board git:(main) x docker run -d \
  --name board-was \
  --network board-net \
  -e DB_HOST=board-db \
  -e DB_PORT=5432 \
  -e DB_NAME=board \
  -e DB_USER=postgres \
  -e DB_PASSWORD=mypassword \
  -p 5001:5000 \
  board-was:1.2
982a704fe7130aa0c6a2ec0e1aff62b5452805dedf7fea6e836c3b87f67939c8
```

WAS 컨테이너 실행

```

● → board git:(main) x docker logs board-was
posts 테이블 준비 완료
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.3:5000
Press CTRL+C to quit

```

WAS 로그

▼ Web 빌드·실행

```

● → board git:(main) x docker build -t board-web:1.1 ./web
[+] Building 1.5s (7/7) FINISHED      docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 114B                      0.0s
=> [internal] load metadata for docker.io/library/nginx  1.3s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                             0.0s
=> [internal] load build context                        0.0s
=> => transferring context: 4.23kB                       0.0s
=> CACHED [1/2] FROM docker.io/library/nginx:alpine@s   0.0s
=> => resolve docker.io/library/nginx:alpine@sha256:d    0.0s
=> [2/2] COPY index.html /usr/share/nginx/html/index.   0.0s
=> exporting to image                                  0.0s
=> => exporting layers                                    0.0s
=> => exporting manifest sha256:2c9f11bfb92375330af32    0.0s
=> => exporting config sha256:da64e23d9359eff36e98b6a    0.0s
=> => exporting attestation manifest sha256:5be56bde6    0.0s
=> => exporting manifest list sha256:45912596007c2181    0.0s
=> => naming to docker.io/library/board-web:1.1          0.0s
=> => unpacking to docker.io/library/board-web:1.1        0.0s
● → board git:(main) x docker rm -f board-web
board-web
● → board git:(main) x docker run -d \
  --name board-web \
  --network board-net \
  -p 8080:80 \
  board-web:1.1
bbf8481eab0af7017a0ae8e01f26e905a3b67e896c825156e1eb5af5ff4d0130

```

Web 이미지 빌드 및 컨테이너 실행

3.2 컨테이너 및 네트워크 확인

- board-web, board-was, board-db 가 모두 Up 상태임을 확인했다.
- 게시글 조회 API는 HTTP 200, 저장 API는 HTTP 201로 정상 응답했다.
- 세 컨테이너가 동일한 board-net 에 연결된 것을 확인했다.

```

● → board git:(main) x docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
bbf8481eab0a   board-web:1.1 "/docker-entrypoint..." 29 seconds ago Up 28 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp board-web
982a704fe713   board-was:1.2 "python app.py"          About a minute ago Up About a minute 0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp board-was
3e5d3aa22274   postgres:17    "docker-entrypoint.s..." About an hour ago Up About an hour 5432/tcp board-db
● → board git:(main) x curl http://localhost:5001/api/posts
[{"content":"\uc548\ub155\ud558\uc138\uc694","created_at":"2026-08-20T06:56:52.156847+00:00","id":1,"title":"\uc548\ub1fd~"}]
● → board git:(main) x docker logs board-was
posts 테이블 준비 완료
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.3:5000
Press CTRL+C to quit
192.168.65.1 - - [20/Aug/2026 06:56:39] "GET /api/posts HTTP/1.1" 200 -
192.168.65.1 - - [20/Aug/2026 06:56:52] "OPTIONS /api/posts HTTP/1.1" 200 -
192.168.65.1 - - [20/Aug/2026 06:56:52] "POST /api/posts HTTP/1.1" 201 -
192.168.65.1 - - [20/Aug/2026 06:56:52] "GET /api/posts HTTP/1.1" 200 -
192.168.65.1 - - [20/Aug/2026 06:57:01] "GET /api/posts HTTP/1.1" 200 -

```

컨테이너 상태, API 호출 및 WAS 로그

▼ Docker 네트워크 상세 확인

```

● → board git:(main) x docker network ls
NETWORK ID     NAME      DRIVER  SCOPE
831d94760084   board-net bridge  local
2a599c06ed94   bridge    bridge  local
a688dab70fde   host      host    local
2545da04da83   none      null    local

```

Docker 네트워크 목록

```
→ board git:(main) x docker network inspect board-net
[
  {
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {},
    "Containers": {
      "3e5d3aa2227498212e7aee81fa88cced902b650dfb0fe8f79c2276014f1203b5": {
        "Name": "board-db",
        "EndpointID": "a2dc7a5bb263d250e79aa382a8ef9036186d453e0edd6af255410d3dfd2d8c9e",
        "MacAddress": "1e:05:77:06:a9:a5",
        "IPv4Address": "172.18.0.2/16",
        "IPv6Address": ""
      },
      "982a704fe7130aa0c6a2ec0e1aff62b5452805dedf7fea6e836c3b87f67939c8": {
        "Name": "board-was",
        "EndpointID": "35040ea50f84cd64597ab1ee04a2f8656631a680ab0c17f38371a804461cc98d",
        "MacAddress": "ea:51:db:21:7d:64",
        "IPv4Address": "172.18.0.3/16",
        "IPv6Address": ""
      },
      "bbf8481eab0af7017a0ae8e01f26e905a3b67e896c825156e1eb5af5ff4d0130": {
        "Name": "board-web",
        "EndpointID": "127c88ae9745127d65282e922293ce0a423e24dcb517c0ecf7711a23b475f0c4",
        "MacAddress": "82:bc:9d:ed:4a:db",
        "IPv4Address": "172.18.0.4/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.18.0.0/16": {
            "IPsInUse": 6,
            "DynamicIPsAvailable": 65530
          }
        }
      }
    }
  }
]
```

board-net 상세 정보

3.3 게시물 저장 및 조회

브라우저에서 게시글을 작성한 뒤 목록에 반영되는 것을 확인했고, PostgreSQL의 `posts` 테이블에서도 동일한 데이터가 저장된 것을 확인했다.



브라우저 게시판 실행 결과

```
○ → board git:(main) x docker exec -it board-db psql -U postgres -d board
psql (17.11 (Debian 17.11-1.pgdg13+2))
Type "help" for help.

board=# SELECT * FROM posts;
 id | title | content |          created_at
-----+-----+-----+-----
  1 | 안 놓 ~ | 안녕 하 세 요 | 2026-08-20 06:56:52.156847+00
(1 row)

board=#
```

PostgreSQL posts 테이블 조회 결과

✓ 검증 결과: 브라우저 → Web → WAS → DB 흐름으로 게시글 저장 및 조회가 정상적으로 동작했다

4. 동료와의 코드 리뷰

받은 리뷰 - 박영신님

은비님의 Dockerfile과 코드들을 읽어본 결과, '코드 리뷰 체크리스트'의 다섯 항목 모두 문제없이 충족되어 있었습니다. 먼저 베이스 이미지는 WAS에 python:3.11-slim, Web에 nginx:alpine을 쓰셨는데 둘 다 용도에 맞으면서 불필요하게 무겁지 않은 선택이었다고 생각합니다. COPY도 필요한 파일만 명시하며 깔끔했습니다. 또한 Web 쪽도 index.html 한 파일만 nginx 문서 루트로 정확히 지정하셨습니다. 포트는 EXPOSE와 실행 옵션이 정확히 맞았습니다. Flask가 실제로 듣는 5000번을 EXPOSE하고 호스트에서는 5001로, nginx는 80을 EXPOSE하고 8081로 매핑하셨는데, 컨테이너 내부 포트와 호스트 포트를 모두 잘 작성하신 것으로 확인됩니다. 환경변수도 app.py의 os.getenv 키(DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD)와 실행 시 넘기는 이름이 그대로 일치했습니다. 마지막으로 컨테이너 이름도 확인해보았을 때, board-db / board-was / board-web으로 역할별로 구분되어 서로 겹치지 않고 이름만 봐도 구성이 읽히게 작성하셨습니다. 이번 종합실습 1에서 요구한 3-tier 구조의 경계와 컨테이너 간 통신 방식을 정확히 작성하신 코드라고 생각합니다.